IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 24 February 2026, accepted 18 March 2026, date of publication 24 March 2026, date of current version 1 April 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3677181

RESEARCH ARTICLE

Integrating Federated Learning and YOLO Models for Robust Object Detection Across Edge Platforms

K. VINOTH AND SASIKUMAR PERIYASAMY, (Senior Member, IEEE)
School of Electronics Engineering, Vellore Institute of Technology, Vellore 632014, India
Corresponding author: Sasikumar Periyasamy (sasikumar.p@vit.ac.in)

ABSTRACT This work proposes an advanced federated learning (FL) framework that utilizes YOLOv8 and YOLOv9 architectures to address the challenges of efficient object detection (OD) in distributed and resource-constrained environments. The framework leverages the Cosine Similarity based Client Metric (CSCM) to optimize client selection and model aggregation, ensuring robust and effective global model updates across Non-Independent and Identically Distributed datasets (Non-IID). Evaluations were conducted using the UniScenes dataset, specifically curated to represent diverse real-world scenarios, encompassing objects such as persons, vehicles, two-wheeler riders, and campus shuttles. The results demonstrate significant improvements in key metrics, including precision, recall, and mean average precision (mAP), while achieving reduced communication and computational overheads compared to conventional aggregation methods like FedAvg. The system's adaptability is further validated through deployment on various edge devices, including Raspberry Pi and NVIDIA Jetson platforms, showcasing efficient real-time OD. The research highlights the potential of federated YOLO models in enabling scalable and high-performance OD for applications such as autonomous systems, surveillance, and IoT-based solutions. Future directions for this research include exploring more complex datasets, expanding the framework's scalability, and enhancing deployment efficiency across heterogeneous edge environments.

INDEX TERMS Client selection, edge computing, federated learning (FL), object detection (OD), cosine similarity, CSCM, YOLO, UniScenes dataset.

I. INTRODUCTION

Object detection (OD), a fundamental task in computer vision (CV), has undergone significant transformation with the introduction of deep learning (DL) based approaches like You Only Look Once (YOLO) [1], [2]. YOLO's real-time capabilities and accuracy have made it a preferred choice for various applications, including autonomous driving and surveillance. However, training robust OD models often requires extensive and diverse datasets, which can raise privacy concerns and pose challenges in resource-constrained environments. Federated learning (FL) provides a viable approach to collaborative model training, allowing distributed devices to share knowledge without compromising sensitive information [3], [4].

Object recognition, a fundamental task in CV, has undergone a significant evolution from classical methods like Viola-Jones [5], which depended on handcrafted features and cascaded classifiers, to the present dominance of deep learning based systems. Early DL algorithms, such as Region-based Convolutional Neural Networks (R-CNN) [6], Fast R-CNN [7], and Faster R-CNN [8], used two-stage architectures, with region proposal development followed by classification. While accurate, these approaches were computationally expensive and lacked real-time functionality. Single-stage detectors, such as YOLO [1], transformed the field by immediately predicting class probabilities and bounding boxes in a single pass, considerably increasing speed. YOLO's popularity sparked a wave of quick

The associate editor coordinating the review of this manuscript and approving it for publication was Szidonia Lefkovits.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

upgrades, with each iteration building on the previous one to enhance accuracy and efficiency. YOLOv3 [9], YOLOv4 [10], YOLOv5, and YOLOv7 [11] improved the architecture by introducing improvements such as better backbone networks, improved loss functions, and superior feature extraction algorithms. This research focuses on YOLOv8 [12] and YOLOv9 [13], the most recent improvements in the YOLO lineage. These models combine several architectural enhancements from the previous version, including better backbone networks, more sophisticated head designs, and loss functions that achieve higher accuracy and speed performance. All these advancements place YOLOv8 and YOLOv9 in the class of the leading OD models that surpass the earlier YOLO versions in accuracy, speed, and many other state-of-the-art object detection methods. This research leverages the strengths of various approaches within a FL framework to overcome the limitations of traditional centralized training methods.

In 2016, Google unveiled FL, a Machine-learning (ML) technique that enables many devices to work together on a model while protecting the privacy of their user data. This strategy is beneficial in dangerous areas where data exchange is prevalent, such as banking, autonomous mobility, and healthcare. However, FL has the same problems as other machine learning methods. Given the increasing interest in FL, [14] looks at its benefits and drawbacks. In FL, Independent and Identically Distributed (IID) data is predicated on the idea that every client device data originates from the same underlying distribution. This indicates that the data's statistical characteristics are consistent across all clients. NonIndependent and Identically Distributed (Non-IID) is when data on many client devices comes from several distributions and may be used in FL. This suggests notable differences in the statistical characteristics of the data between clients. FL enables edge devices, including smartphones and Internet of Things (IoT) devices, to collaboratively develop a predictive model while keeping their training data localized. However, this strategy encounters a statistical problem when local data is Non-IID.

This classic FL architecture, illustrated in Figure.1, enables distributed ML, training a shared global model across multiple decentralized clients while preserving data Locality without sharing raw data [15], [16]. Each client, being an edge device with its data and processing, holds a dataset that prevents sensitive information from being centralized. Clients can independently train a local model starting from an initial global model from the FL central server concerning datasets across each round. It will be adapted to the peculiar data characteristics of the client based on this local training phase; however, it is represented as different neural networks in each client block. After local training, clients send only model updates, i.e., the changes in parameters, not the raw data, to the FL server [17]. The dotted lines connecting the clients to the server represent these updates. The server combines the updates from each device using federated averaging (Fedavg), resulting in a

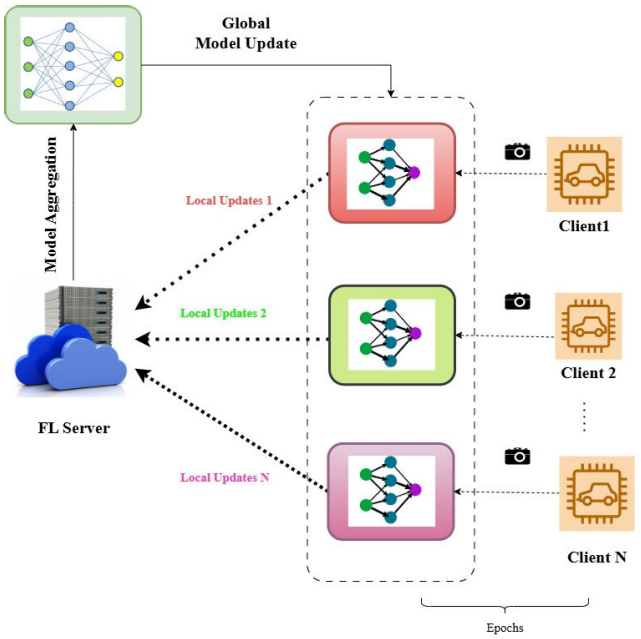


FIGURE 1. Architecture - FL.

refined global model. The global model update is then distributed to each client to start the next round, iteratively refining the model through multiple rounds of training. This architecture effectively balances model accuracy with the critical need for data privacy by keeping sensitive data decentralized and limiting the information transmitted to the server to model updates only [18]. The process efficiently leverages distributed resources without compromising the confidentiality of individual client datasets.

To overcome the inherent challenges with FL, such as those posed by the requirement for efficient client selection and model aggregation in Non-IID, we introduce the CSCM. CSCM prioritizes updates with strong cosine similarity, optimizing the FL process for improved efficiency and performance, thus achieving faster convergence, better accuracy, and lower communication overhead, especially in diverse and potentially conflicting data distributions across clients. By adopting a focused strategy, this approach optimizes the utilization of limited resources for FL on Non-IID data, ultimately enhancing the process's resilience.

The performance of the presented FL framework, using YOLOv8 and YOLOv9 architectures, was rigorously tested by deploying the trained models onto a wide variety of edge devices with diverse computational capabilities and resource constraints [19]. This multidimensional approach checked the framework's scalability, adaptability, and real-time performance in practical deployment scenarios. The edge devices were selected, representing different processing power and resource availability tiers. It was thus made to ensure complete evaluation across a realistic range of deployment environments. It included the very affordable and easy-to-use resource-constrained Raspberry Pi 5 model as a benchmark

for minimalist resource scenarios that can be used to assess the performance of models in rather extreme limitations of processing power and memory. Benchmark performance was reported in [20]. The Nvidia Jetson Nano was used as a mid-range edge device, which was a good representative for use in our evaluation, being relatively affordable and offering significantly better performance than the Raspberry Pi 5, making it a very popular platform to deploy AI applications at the edge. Finally, the high-end Nvidia Jetson AGX Xavier was implemented to test performance under optimum resource conditions. This had to be provided with significant capabilities of both GPU and memory to evaluate the models' training without significant restrictions on resources to present an idea about what could become possible under higher computational abilities [21], [22]. For each of these edge devices, two key performance metrics were collected and analyzed meticulously: inference time (latency) and frames per second (FPS), with a methodology similar to [23]. Inference time, measuring the time required to process a single image or frame, directly reflects real-time processing capability and responsiveness. The FPS metric is computed as the reciprocal of inference time. It provides an aggregate measure of throughput, which is important in determining the system's suitability for real-time applications such as autonomous driving and video surveillance. This multi-edge diversity with careful performance measurements of different edge devices offers a good, robust evaluation of the practical applicability of the FL framework proposed here across a wide range of resource-constrained settings. The results from this rigorous testing provide invaluable insights into the framework's scalability and suitability for various real-world applications. Two deep learning models for drone detection, YOLOv7 and YOLOv8, are compared in this [24]. In crucial security applications, the YOLOv8 model seems more promising and may help with real-time drone identification.

This work presents a novel FL framework integrating state-of-the-art YOLO architectures to address OD challenges in distributed and resource-constrained environments.The proposed framework introduces a Cosine Similarity–based Client Metric that enables selective client participation and more efficient model aggregation, which reduces the communication cost and computational overhead without degrading the detection performance. Moreover, the proposed framework will also be evaluated from a deployment-centric perspective, showcasing real-time feasibility on heterogeneous edge platforms through TensorRT and FP16 optimization. Together, these contributions distinguish the proposed approach by jointly addressing aggregation efficiency, scalability, and practical edge deployment within a unified federated learning framework.

The contributions of the Proposed work:

- We introduce a Cosine Similarity–based Client Metric (CSCM) for federated object detection that improves aggregation efficiency under Non-IID data distributions through selective client participation.
- We demonstrate that the proposed framework achieves improved accuracy, efficiency trade-off, significantly reducing communication and computational overhead compared to FedAvg while maintaining competitive detection performance.
- We integrate YOLOv8 and YOLOv9 within a unified federated learning framework and provide a comprehensive evaluation across multiple training configurations.
- We validate the practical deployability of the proposed approach through real-time inference experiments on heterogeneous edge devices using TensorRT and FP16 optimization.

II. RELATED WORKS

The Literature Review section presents an exhaustive examination of the current state of knowledge, synthesizing key findings and insights from relevant studies that inform and contextualize this research. It highlights the advancements and challenges in integrating FL with OD, particularly in resource-constrained environments.

Identification and localization of objects inside digital images or videos constitute the core task of OD in CV. Its use has benefited numerous fields, especially robotics, surveillance, autonomous driving, and medical image processing. In contrast to conventional cloud-based computing, Edge computing processes data locally, lowering latency and speeding up reaction times. Due to the restricted processing resources, power limits, and privacy considerations, OD for edge devices poses special issues. Complex DL models are challenging to execute on edge devices because they often have lower processing units and less memory than cloud servers [25]. FL presents a viable way to address these issues. It also maintains data privacy by training models locally on individual devices and aggregating their weights to look into the viability of FL for ML applications, namely image-based datasets for OD. Substantial challenges are posed by data labeling, model training, data rate optimization, ensuring system reliability, minimizing transmission overhead, and managing resources efficiently. Future directions for FL research in-vehicle networks are also outlined in [26] to overcome the shortcomings of previous FL research, which frequently uses synthetic partitions and open datasets and presents a collection of actual images. The open dataset is a useful tool for creating object identification algorithms and evaluating the effectiveness of FL systems in vehicular network contexts. It comprises over 900 images from 26 street cameras, covering seven object categories. FL has been investigated for OD in self-driving car systems, proving its feasibility and possible advantages [27]. A prototype FL system was developed, and by using secure weight aggregation and local model training, it achieved a mean average precision(mAP) of 68%. Driver activity identification (DAR) poses special issues because of finer activity variations and increased privacy concerns, even though video action identification has been examined extensively. These problems are resolved by FL's decentralized strategy,

which permits collaborative learning without jeopardizing data security. An FL-based DAR model is proposed [28], and its performance is evaluated using two datasets. Findings show that FL benefits DAR applications with competitive performance in centralized and decentralized contexts.

Integrating advanced technology with Connected and Autonomous Vehicles (CAVs) greatly improved driving assistance systems. However, CAVs are vulnerable to security risks due to the exchange and transmission of sensitive data. FL solves this problem by creating collaborative models while preserving local data privacy [29].

FL has also been used in vehicular networks to develop federated deep neural networks that improve human perception in open-road environments while reducing communication costs. Software platforms and contrastive sensor layout techniques have been designed to address resource distribution challenges, though they remain a focus for ongoing research [30]. With more interest in FOD, numerous studies show its potential for higher detection accuracy in adverse scenarios, including inclement weather [31], [32]. To achieve greater accuracy and scalability, multi-layered frameworks incorporating data from roadside units and CAVs have been suggested [33]. These advancements highlight FL as a viable strategy for improving the accuracy and security of autonomous vehicle systems. Future investigations should target unresolved challenges and explore innovative strategies to enhance FL's efficacy in this domain. Progress has been made in real-time activity recognition, including edge-based applications essential to computational resource constraints. This paper presents a real-time elderly activity recognition system that employs the OD algorithm YOLO on Jetson Nano, thus bridging the noticeable void in hardware-based elderly activity recognition [34]. For the study, two datasets were developed: a custom dataset that integrated three public datasets and a new elderly activity recognition dataset based on the physical characteristics of individuals, namely their walking styles and overall movements.

Among the on-deployment assessments made using the three algorithms, namely YOLO, Faster R-CNN, and SSD, YOLO recorded the highest speed in inference while still performing well in other areas. It was noted attention accuracy was, however, higher in the walking activities captured by the elderly dataset when compared to the custom dataset, and this was due to gait disparity across the population. The evaluation focused on accuracy and other metrics, including inference speeds, resource utilization, and energy efficiency, with the trade-offs between computation cost and detection precision [35]. Additionally, the research outlines model pruning, quantization, and lightweight architectures, among other techniques aimed at optimizing OD models for deployment on edge devices such as GPUs and CPUs. These methods decreased the computational requirements, which were crucial for real-time purposes such as assisted living and autonomous navigation applications [36]. To label all instances of a category within an image, a process known as Simultaneous Detection and Segmentation (SDS). It focuses on pixel-level segmentation which requires identifying specific object instances instead of merely classifying them. Reference [37] propose a new architecture for SDS that builds upon previous convolutional neural networks designed for category-independent region proposals (R-CNN). Reference [38] utilizing a multi-region deep convolutional neural network (CNN) designed for semantic segmentation. This CNN-based approach enhances localization sensitivity, ensuring precise object localization while capturing diverse discriminative appearance features. By integrating recognition module within an iterative localization process balancing box proposal scoring with precise placement adjustments through a deep CNN regression model. Recent research involves the application of federated learning in the area of autonomous vehicles and edge-based object detection. Several research endeavors have been conducted on the application of FL-based object detection and perception in the context of connected autonomous vehicles, tackling various issues related to deployment. For instance, the application of federated object detection in the context of connected autonomous vehicles was studied in [39], involving the evaluation of various state-of-the-art object detection networks. Studies like FedPylot investigate the FL-based optimization of the YOLO algorithm in the context of the Internet of Vehicles paradigm [40], and the presentation of AutoFed includes multimodal FL in the attempt to achieve robust perception in the context of autonomous vehicles [41]. Similarly, the application of FL in the area of semantic segmentation of routes as well as objects in Avs is presented in [42], as well as various FL-related discussions in the area of transport and edge computing [43]. Autonomous vehicles, provide safer, more efficient, and convenient transportation. A novel multi-sensor fusion and segmentation technique for multi-object tracking in self-driving cars is proposed, utilizing Deep Q Network (DQN). The strategy employs noise reduction, contrast enhancement, and adaptive thresholding methods. The YOLO V7 model is used for detection and classification [44].

III. PROPOSED SYSTEM

A. FRAMEWORK AND METHODOLOGY OF FL FOR OD

The proposed system employs both the YOLOv8l and YOLOv9c models and state-of-the-art advancements of the YOLO architecture, each utilized in separate research cases to evaluate their unique strengths and detection capabilities.The YOLO excels at concurrently detecting multiple objects within an image by predicting bounding boxes and their corresponding class probabilities. This integrated detection architecture provides the necessary speed and accuracy for applications that demand real-time insights from visual data, such as self-driving cars or security systems. YOLOv8l and YOLOv9c are chosen as the backbone models to effectively illustrate the trade-off between detection accuracy and model complexity in a federated learning context. Although lighter variants such as YOLOv8m have fewer parameters, preliminary experiments indicated reduced

robustness under Non-IID data partitioning, particularly for small and occluded objects. In contrast, larger models such as YOLOv9e incur significantly higher communication overhead due to increased memory synchronization requirements, making them less suitable for resource-constrained edge environments.

Recent YOLO versions (e.g., YOLOv11 and YOLOv12) and transformer-based detectors such as RT-DETR were not considered in this study in order to ensure architectural stability and fair comparison within the federated learning framework. These models typically introduce increased architectural complexity and memory demands, which can further exacerbate federated training overhead. Nevertheless, the proposed CSCM-based framework is model-agnostic, and extending the current approach to lighter YOLO variants, newer YOLO versions, or transformer-based detectors constitutes a promising direction for future research.

The system's workflow begins with initializing a global model using weights from a pre-trained YOLO model. This global model acts as a baseline and is shared across all client nodes where local datasets reside. Using a pre-trained model as a starting point accelerates convergence and improves initial performance, particularly when the clients possess limited computational resources or data volumes. Moving forward, the emphasis is on preparing client data for distributed training. In this framework, each client represents a distributed data source, and the system simulates random distribution of data subsets to mimic real-world data variety encountered across different locations. By selecting a representative subset of files, each client effectively processes what it deems most appropriate, ultimately contributing to the richness of the collective training data.

The FL architecture utilizes local training as a fundamental building block. Clients use their prepared datasets to train local models, starting from a copy of the global model. The local training routine is comprehensive, encompassing multiple epochs to ensure that the model sufficiently learns from the local data variations. Given that each client's dataset may differ significantly in terms of class distribution, size, and image complexity, having local models adapt their learning according to these factors is crucial. This local adaptability enhances the model's robustness as varying data distributions across clients contribute unique insights into the model's learning process. The clients' raw data remains on-device after the training phase, without being sent to a central server. Instead, they transmit updates of their model weights. These updates encapsulate the learning acquired from the local data without exposing. The system then employs a client selection mechanism illustrated in Figure 2 based on the CSCM to determine which model updates are most beneficial for the global model. By evaluating the similarity of parameter changes, this metric ensures that only the most informative and relevant contributions are aggregated. The aggregation process assimilates the selected updates into the global model, synthesizing them to enhance its predictive performance iteratively.

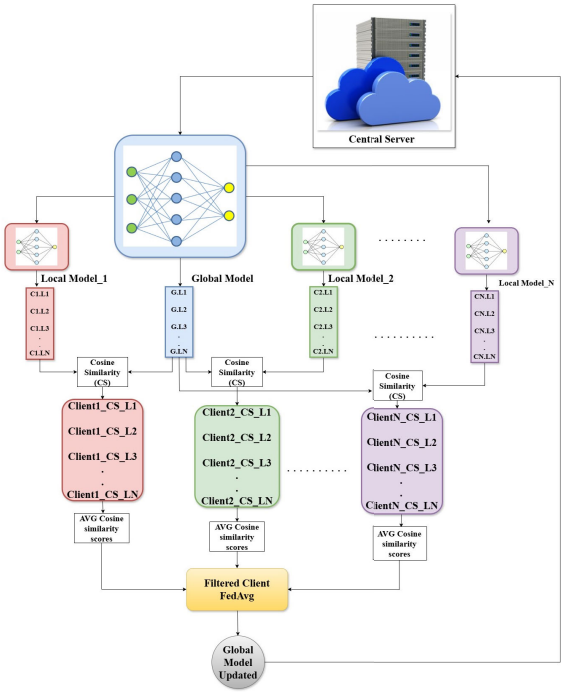


FIGURE 2. Framework of CSCM for FL client contributions.

B. CLIENT SELECTION AND MODEL AGGREGATION STRATEGY

In FL, the strategic selection of clients and aggregation of models are vital components, especially when tackling intricate tasks such as OD. One of the critical challenges in FL is deciding which client updates should be integrated into the global model to ensure coherent and beneficial progression.

CSCM is designed to quantitatively assess the contributions of each client's model updates through a comparison mechanism that evaluates the alignment between the client's locally trained model and the aggregated global model. Specifically, It utilizes cosine similarity to assess the similarity between the client model's parameters and those of the global model. At each federated communication round, the cosine similarity measure is computed in a layer-wise manner using the client model update vectors. For each client, the layer-wise cosine similarity values are averaged to obtain a single similarity score. Clients are then ranked based on this score, and a top-$p\%$ client selection criterion is applied. In all experiments, the selection ratio is fixed to $p = 50\%$, and the selected clients participate in model aggregation, directly determining the communication and aggregation complexity.This evaluation metric is calculated for each model layer by assessing the vector alignment of the client's parameter updates with those of the global model. The cosine similarity metric, used to quantify the similarity between two vectors, the parameter vector of a client's model (W_{client}) and the global model (W_{global}). It is expanded as

At each federated round t, every participating client i initializes its local model with the current global model

$W^{(t-1)}$ and performs local training for E epochs on its private dataset. After local training, the client computes its model update (delta) as

$$\Delta W_i = W_i^{(t)} - W^{(t-1)}. \quad (1)$$

To evaluate the alignment between client updates and the global optimization direction, a layer-wise cosine similarity is computed between the client update ΔW_i and the aggregated global update direction $\Delta W^{(t-1)}$. For each network layer l, the cosine similarity is defined as

$$\cos_l(\Delta W_i, \Delta W^{(t-1)}) = \frac{\Delta W_{i,l} \cdot \Delta W_l^{(t-1)}}{\left\|\Delta W_{i,l}\right\| \left\|\Delta W_l^{(t-1)}\right\|}. \quad (2)$$

The final client contribution score is obtained by averaging the cosine similarity values across all L layers:

$$S_i = \frac{1}{L} \sum_{l=1}^{L} \cos_l . \quad (3)$$

Clients are then ranked based on their contribution scores S_i, and the top $p\%$ of clients are selected for aggregation in the current round. The selected client updates are aggregated using weighted federated averaging to update the global model.

This metric provides a normalized measure of change alignment, essential in deciding how each client's contribution aligns with the global optimization trajectory. Once the CSCM values are determined for each client, the strategy identifies the most valuable client updates to optimize model performance. This step is crucial because not all client updates are beneficial, and indiscriminate aggregation could lead to model drift, where the global model diverges from optimal performance due to conflicting updates. The system uses a threshold-based approach, selecting a percentage of clients with the highest similarity scores. This selection criterion ensures that only harmonized and complementary knowledge is synthesized into the global model, thus maintaining or enhancing its performance at each round.

Another cornerstone of the strategy is the process of aggregating client contributions. Once the selected clients are determined based on CSCM, their model updates are averaged and used to update the global model. This averaging process helps smooth out the noise and variance of individual data idiosyncrasies at each client site. Techniques such as Fed averaging can be employed, where clients contributing larger datasets or those with higher CSCM scores receive greater weight, thus subtly introducing an additional layer of refinement in the aggregation process. The overarching goal of model aggregation is to ensure incremental performance improvements of the global model. By carefully filtering and integrating client updates, the system benefits from diverse data patterns evident in distributed datasets without compromising on qualitative learning or fostering negative learning behaviors.

C. DEPLOYMENT OF FL MODELS ON EDGE DEVICES

Deploying FL models on edge devices involves optimizing model performance to meet edge environments' resource constraints and operational demands. After federated training, the YOLOv8 and YOLOv9 models generate a best.pt file, representing the trained PyTorch models. These models are converted to Open Neural Network Exchange (ONNX) format, resulting in best.onnx files, a versatile format compatible with multiple platforms and tools. The Tensor Runtime (TensorRT) framework is used to optimize these ONNX models for deployment, producing highly efficient models tailored for edge computing. The optimized models are deployed and tested on various edge devices, including NVIDIA AGX Xavier, NVIDIA Jetson Nano, and Raspberry Pi 5 representing a range of computational capabilities commonly used for edge AI tasks. The deployment process involves setting up the edge devices, loading the optimized ONNX models, and preparing a test dataset simulating real-world OD scenarios. Each device's inference speed is evaluated by measuring the latency (time taken to process a single image or frame) and FPS, which are critical for real-time performance. For Raspberry Pi devices, inference runs on the CPU due to the lack of a dedicated GPU. At the same time, Jetson Nano uses its onboard CUDA cores for efficient execution, and AGX Xavier leverages its advanced GPU and tensor cores for high throughput. Performance metrics such as latency and FPS are recorded and compared across devices, analyzing each platform's trade-offs between computational efficiency and model accuracy. This deployment approach ensures that FL models, trained in a distributed environment, are effectively adapted for edge devices, demonstrating their adaptability and readiness for real-world applications like surveillance, autonomous driving, and IoT systems. The evaluation highlights the feasibility of deploying advanced OD models on resource-constrained devices, bridging the gap between centralized AI training and edge AI deployment.

IV. EVALUATION AND EXPERIMENTAL RESULTS

A. DATASET DESCRIPTION

For this study, we developed a custom dataset named UniScenes, specifically tailored to capture the unique environment of our university campus. The dataset comprises 1,000 images collected from various campus locations to ensure a diverse representation of the target objects and surroundings. This deliberate design reflects the Non-IID characteristics inherent in FL scenarios. The dataset encompasses four distinct classes: persons, cars, two-wheeler riders, and campus shuttles. These categories represent the most common mobile entities within the university ecosystem, providing a comprehensive view of campus traffic and pedestrian activity. A range of preprocessing and data augmentation strategies were implemented to strengthen the model's ability to generalize. Preprocessing steps included auto-orientation correction and conversion to grayscale, which standardized the input data and reduced computational



FIGURE 3. Diverse Images from UniScenes dataset.

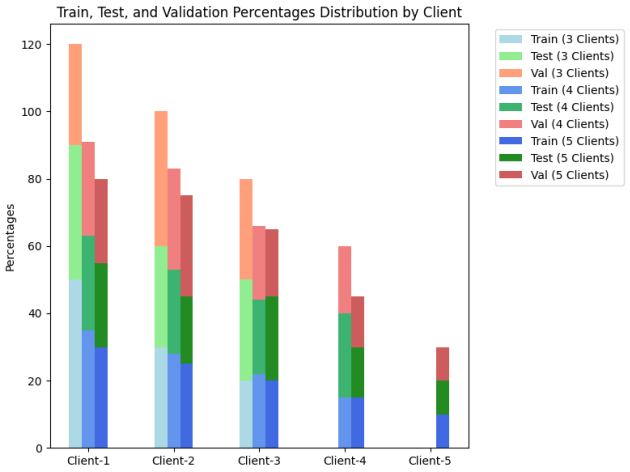


FIGURE 4. Client-specific data allocation: training, testing, and validation percentages illustrating Non-IID.

TABLE 1. Non-IID training data distribution across clients (C = 3).

Client ID	Training Images Assigned	Training Set (2200)
Client 1	1100	50%
Client 2	660	30%
Client 3	440	20%
Total	2200	100%

complexity. A consistent image size of 640 × 640 pixels was established across the dataset through resizing. Data augmentation significantly enhanced the dataset's size and diversity, leading to improved model generalizability. Table 1 shows how the augmented training data was divided among the three clients. The full dataset contains 1000 images. Out of these, 700 images were first set aside for training. Data augmentation was applied only to those 700 images. After augmentation, the training set increased to 2200 images.

To create a Non-IID setting, the training data was not split equally. Instead, we used a 50/30/20 split. Client 1 received 1100 images, Client 2 received 660 images, and Client 3 received 440 images. Because the clients hold different amounts of data, their local updates are naturally different. In addition, each client was assigned a dominant object class, so the class distribution varies from one client to another. A fixed random seed was used to ensure that the split can be reproduced. The remaining 200 validation images and 100 test images from the original dataset were retained at the central server and were not distributed to clients. These datasets were used exclusively for global model evaluation to ensure fair and consistent performance comparison across federated configurations. No augmentation was applied to the validation and test sets to prevent data leakage and preserve evaluation integrity.The dataset's diversity is illustrated in Figure 3, which showcases diverse images from the UniScenes Dataset. Table 1 reports Non-IID Training Data Distribution Across Clients (C = 3). Figure 4 extends this analysis to scenarios with client 3 (C=3), 4 (C=4) and 5 (C=5) clients, demonstrating the versatility of the dataset's Non-IID properties. This uneven data distribution mirrors real-world FL environments, where clients typically possess varying amounts of data.

B. KEY METRICS AND EVALUATION

A comprehensive evaluation framework is employed to assess the efficacy and efficiency of FL models. Through a multidimensional analysis, actionable insights are delivered into key performance indicators, encompassing model precision, recall, mAP, complexity and scalability. The evaluation metrics comprise model performance (precision, recall, and mAP), model complexity (total parameters and computational complexity per client), scalability (communication cost per round and total communication cost). By examining these metrics, a nuanced understanding of FL model strengths, weaknesses, and optimization opportunities is obtained.The experiments were conducted using the Adam optimizer, with an initial learning rate of 0.001 and a cosine learning rate decay schedule. The batch size was fixed at 16 per client for all training runs. Local training was performed for a varying number of epochs E 4,6,8, while the number of federated communication rounds was varied as R 4,5,6 across different experimental configurations. All clients were assumed to be available at each round, after which the proposed CSCM-based client filtering mechanism selected a subset of clients for aggregation. To enhance generalization and robustness, a consistent data augmentation pipeline was employed across all clients, including random horizontal flipping, image scaling, and resizing to 640×640 resolution, Mosaic augmentation, and HSV-based color jittering.

1) PRECISION

The Precision metrics evaluates the ratio of correct positive predictions to the total number of positive predictions generated by the model. This metric evaluates the model's effectiveness in identifying relevant instances accurately, reflecting its accuracy and dependability [45]. Precision is calculated by:

$$Precision = \frac{T_p}{T_p + F_p} \tag{4}$$

where, True Positives (T_p): Correctly identified instances, False Positives (F_p): Incorrectly identified instances

2) RECALL

The recall metric assesses the model's ability to detect actual positive instances, measuring the ratio of correctly identified positives to the total number of positive cases [46]. This provides insight into the model's detection accuracy.

$$Recall = \frac{T_p}{T_p + F_N} \tag{5}$$

3) mAP AND IoU

The mean average precision (mAP) is a standard metric used to evaluate object detection performance across multiple classes. It is computed as the mean of the average precision (AP) scores for each class, where AP is the area under the precision-recall curve for that class. Mathematically:

$$mAP = \frac{1}{C} \sum_{i=1}^{C} \mathrm{AP}_i \tag{6}$$

where C is the number of object classes, and APi is the average precision for the ith class.

Intersection over Union (IoU) is a threshold-based metric used to determine whether a predicted bounding box is considered a correct detection. It is calculated as the ratio of the area of overlap between the predicted bounding box and the ground truth box to the area of their union:

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}} \tag{7}$$

In our evaluation, we use mAP@50, which denotes the mean average precision when the IoU threshold is fixed at 0.5. This means a predicted object is considered a correct detection if the IoU with a ground truth box is ≥ 0.5.

4) TOTAL PARAMETERS

The total number of parameters in the global model indicates its complexity and size. For our training setup, we considered the aggregated parameters from all participating clients that contributed to the training process, highlighting the scope of data being processed and transmitted [47]. significantly experiments revealed that the YOLOv8l model comprises 43,636,008 parameters, while the YOLOv9c model has 25,532,316 parameters. To calculate the overall parameter, count for each training round, we multiplied the total number of model parameters by the number of selected clients that provided updates. It is given by

$$Total\ Parameter = \sum_{i=1}^{n} P_i\ Num\ of\ clients \tag{8}$$

where Pi is the number of parameters in the model for client i, and n is the number of selected clients participating in that round.

5) COMMUNICATION COST

This metric evaluates the amount of data transmitted between client nodes and the central server during the model update process [48]. The communication cost is often represented in bytes, indicating the amount of data transmitted per round of updates. This metric is computed by considering the number of participating clients and the aggregate amount of model parameters transmitted and received by each client, with each parameter represented as a 4-byte floating-point value.

$$\begin{aligned} &Communication\ Cost\ per\ Round(bytes) \\ &\quad = Total\ Parameters \\ &\qquad \times\ Number\ of\ Selected\ Clients \times\ 4 \end{aligned} \tag{9}$$

where Total Parameters is the number of parameters in the model, Number of Selected Clients is the number of clients contributing updates in a given round, and 4 represents the size (in bytes) of a 32-bit floating-point parameter.

Total Communication Cost: This metric represents the cumulative communication costs incurred across all rounds.

$$\begin{aligned} &Total\ Communication\ Cost \\ &\quad = \sum_{r=1}^{R} Communication\ Cost\ Per\ Round \end{aligned} \tag{10}$$

where R is the total number of communication rounds, and r denotes the index of each communication round.

6) COMPUTATIONAL COMPLEXITY

These metric gauges the computational effort required at the client level to train local models and at the server level to integrate updates [49], [50]. It is usually expressed in terms of the number of operations (e.g., floating point operations per second). The computational workload is calculated by multiplying the number of data samples, model parameters, and training epochs per client, and then summing these values across all clients to obtain the total computational effort per round, which encompasses local data processing and model update costs. It is given as

$$\begin{aligned} &Computational\ Complexity\ per\ Client \\ &\quad = N \times Total\ Parameter \times E \end{aligned} \tag{11}$$

where N is the number of local data samples per client, Total Parameters is the number of model parameters, and E is the number of local training epochs.

Total Computational Complexity: This metric is calculated by aggregating the computational complexity of each client,

multiplied by the number of participating clients, across all rounds

$$\begin{aligned} &\textit{Total computational complexity} \\ &= \sum_{r=1}^{R} \textit{Computational Complexity per Client} \\ &\times \textit{Num of Filtered Clients} \end{aligned} \tag{12}$$

where R is the number of total rounds, and Number of Filtered Clients is the number of clients selected based on the CSCM filtering mechanism in each round.

C. EXPERIMENT WITH YOLOv8

This section details the experimental evaluation of the YOLOv8 model, focusing on its application for OD in distributed settings. The experiments explore settings, including FL scenarios, to evaluate performance across Non-IID datasets. Results emphasize YOLOv8's adaptability and balancing efficacy with efficiency in resource-limited settings while ensuring the scalable performance.

The results of FL with YOLOv8 across various configurations are presented in Table 2. The table comprehensively compares the CSCM and the FedAvg methods in terms of communication cost, computational complexity, precision, recall and mAP50 across different client (C), epoch (E), and round (R) configurations. A detailed analysis of the results reveals that the CSCM method consistently outperforms the FedAvg method across all configurations. Specifically, with configuration C=3, E=4, R=4, the CSCM method achieves a significant reduction in communication cost, with a value of 0.70 Giga bytes compared to 2.09 Giga bytes for the FedAvg method Similarly, the CSCM method achieves a lower computational complexity of 698 GFLOPs, compared to 6283 GFLOPs for the FedAvg method. Furthermore, the CSCM method achieves high mAP50 values across all configurations, demonstrating its effectiveness in OD tasks. Specifically, it achieves mAP50 values of 0.796 (Figure 5), 0.812 (Figure 6), and 0.829 (Figure 7) for configurations C=3, E=4, R=4, C=4, E=6, R=5, and C=5, E=8, R=6, respectively. In addition to its superior communication cost, computational complexity, and mAP50 values, the CSCM method also achieves higher precision, recall, and mAP50 values than the FedAvg method. This showcases the efficacy of the CSCM method in improving the accuracy and efficiency of FL.

TABLE 2. Results of FL with YOLOv8 across various configurations.

Config	Type	CC (GB)	TCC	Prec	Recall	mAP 50
C=3, E=4, R=4	FedAvg	2.09	6283	0.838	0.691	0.77
C=3, E=4, R=4	CSCM	0.70	698	0.865	0.696	0.796
C=4, E=6, R=5	FedAvg	3.49	20944	0.905	0.681	0.80
C=4, E=6, R=5	CSCM	1.75	5236	0.812	0.754	0.812
C=5, E=8, R=6	FedAvg	5.24	52360	0.828	0.726	0.806
C=5, E=8, R=6	CSCM	2.09	8378	0.738	0.784	0.829

Config: Configurations, CC: Overall Communication Cost (GB), TCC: Total Computational Complexity (GFLOPS), Prec: Precision

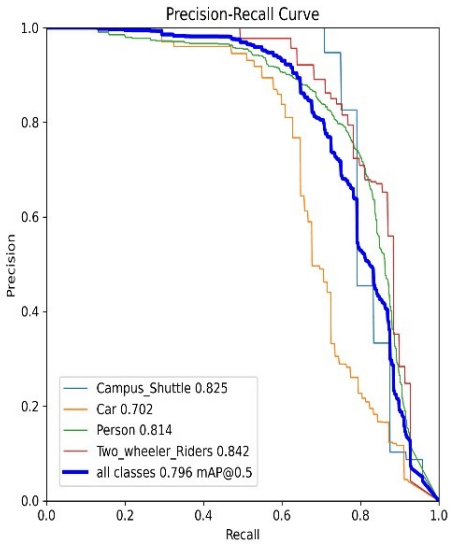


FIGURE 5. mAP50 values for CSCM with configuration C = 3, E = 4, R = 4.

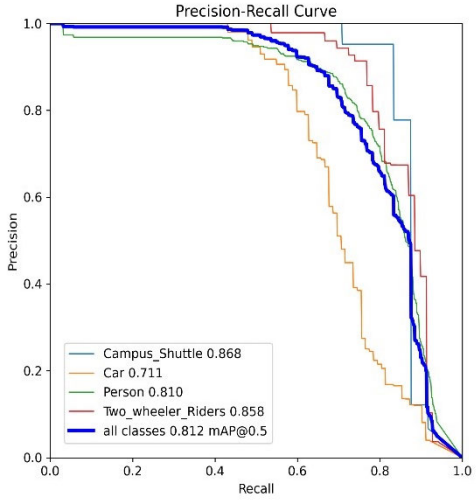


FIGURE 6. mAP50 values for CSCM with configuration C = 4, E = 6, R = 5.

D. EXPERIMENT WITH YOLOv9

This section presents the experimental analysis of the YOLOv9 model, an advanced iteration in the YOLO series designed to enhance OD capabilities in complex environments. YOLOv9 builds upon its predecessors by incorporating improved contextual understanding and precision, making it well-suited for scenarios requiring detailed recognition in cluttered or dynamic scenes. The experiments evaluate the model's precision, recall, and mAP across multiple configurations. YOLOv9's integration into FL frameworks is also examined, highlighting its ability to maintain accuracy while optimizing communication and computational costs. These evaluations demonstrate YOLOv9's potential to balance performance and resource efficiency in diverse deployment scenarios.

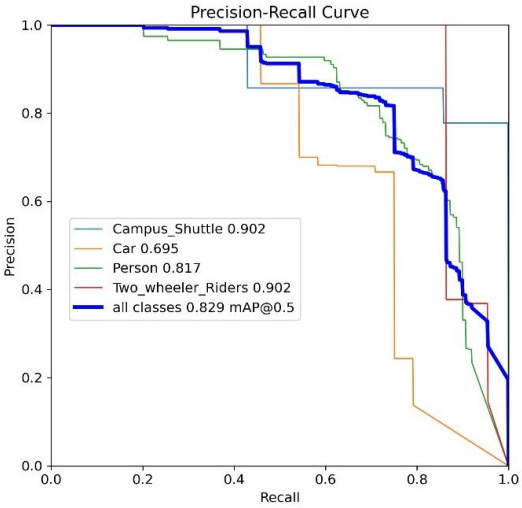


FIGURE 7. mAP50 values for CSCM with configuration C = 5, E = 8, R = 6.

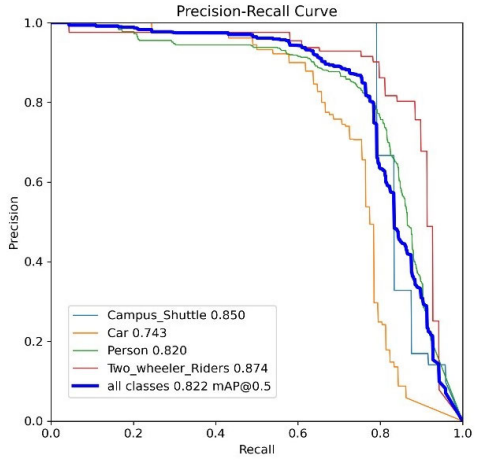


FIGURE 8. mAP50 values for CSCM with configuration C = 3, E = 4, R = 4.

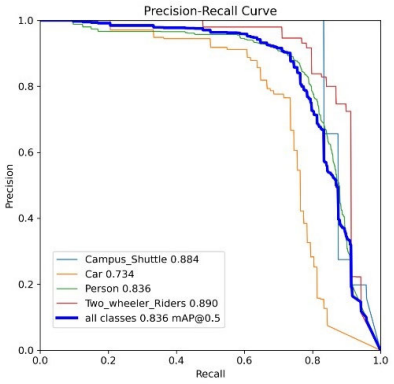


FIGURE 9. mAP50 values for CSCM with configuration C = 4, E = 6, R = 5.

The results of FL with YOLOv9 across various configurations are presented in Table 3. This table comprehensively compares the CSCM and the FedAvg methods, highlighting

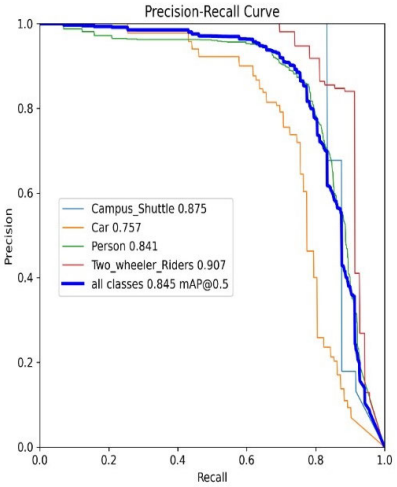


FIGURE 10. mAP50 values for CSCM with configuration C = 5, E = 8, R = 6.

TABLE 3. Results of FL with YOLOv9 across various configurations.

Config	Type	CC (GB)	TCC	Prec	Recall	mAP 50
C=3, E=4, R=4	FedAvg	1.23	3677	0.814	0.731	0.814
C=3, E=4, R=4	CSCM	0.41	409	0.852	0.778	0.822
C=4, E=6, R=5	FedAvg	2.04	12256	0.868	0.731	0.819
C=4, E=6, R=5	CSCM	1.02	3064	0.852	0.778	0.836
C=5, E=8, R=6	FedAvg	3.06	30639	0.903	0.746	0.833
C=5, E=8, R=6	CSCM	1.23	4902	0.860	0.757	0.845

Config: Configurations, CC: Overall Communication Cost (GB), TCC: Total Computational Complexity (GFLOPS), Prec: Precision

their differences in communication cost, computational complexity, precision, recall, and mAP50 values. Upon examining the results, it becomes clear that the CSCM method surpasses the FedAvg method in all configurations. For example, in configuration C=3, E=4, R=4, the CSCM method reduces the cost of communication by 66.7% compared to the FedAvg method, achieving a value of 0.41 GB. Additionally, the CSCM method demonstrates a substantial decrease in computational complexity, with a value of 409 GB Floating Point Operations Per Second (GFLOPs) compared to 3677 GFLOPs for the FedAvg method. The CSCM method also exhibits exceptional performance in OD tasks, achieving high mAP50 values across all configurations. Specifically, it achieves mAP50 values of 0.822 (Figure 8), 0.836 (Figure 9), and 0.845 (Figure 10) for configurations C=3, E=4, R=4, C=4, E=6, R=5, and C=5, E=8, R=6, respectively. Regarding precision, recall, and mAP50 values, the CSCM method consistently outperforms the FedAvg method. This illustrates the benefits of the CSCM method in enhancing the accuracy and efficiency of FL. The YOLOv9 model trained using the CSCM method demonstrates its effectiveness in OD use cases, demonstrated in Figure 11. This visual representation of OD on the UniScenes dataset highlights the CSCM method's ability to achieve accurate OD while minimizing communication cost and computational complexity.

TABLE 4. Quantitative comparison of centralized and federated methods.

Method	Comm. Cost (GB)	Precision	Recall	mAP50	mAP Gain (%)	Comm. Reduction (%)
SDS [37]	–	0.52	0.48	0.496	–	–
R-CNN BB [6]	–	0.55	0.51	0.533	–	–
MR_CNN_MORE_DATA [38]	–	0.76	0.72	0.739	–	–
FedPylot [52]	–	–	–	0.68	–	–
Fed Dynamic DP [53]	–	0.80	0.78	0.75	–	–
FedProx [54]	–	–	–	0.68	–	–
SCAFFOLD	–	–	–	0.56	–	–
FL-YOLOv3 [16]	55.31	0.821	0.792	0.831	−0.48	–
FL-Faster R-CNN [9]	76.6	0.835	0.805	0.842	−0.36	–
FedAvg (IID)	6.502	0.862	0.785	0.835	Baseline	Baseline
VINO_EffiFedAV (IID)	2.926	0.864	0.777	0.843	+0.96	55.0
FedAvg (Non-IID)	6.502	0.815	0.532	0.603	Baseline	Baseline
VINO_EffiFedAV (Non-IID)	2.113	0.626	0.597	0.616	+2.15	67.5
CSCM–YOLOv8 (Ours)	2.09	0.738	0.784	0.829	+37.5*	67.8
CSCM–YOLOv9 (Ours)	1.23	0.860	0.757	0.845	+40.1*	81.1



FIGURE 11. Visual Representation of OD on the UniScenes dataset using YOLOv9.

Table 4 provides a quantitative comparison of the proposed CSCM framework with centralized, federated, and state-of-the-art object detection models in terms of precision, recall, mAP50, and total communication cost. It also includes additional federated optimization strategies such as FedPylot, Fed Dynamic DP, FedProx, and SCAFFOLD.Both IID and Non-IID federated distribtions are considered to highlight the effects of data heterogeneity. Percentage gains in mAP50 and communication reduction are computed with respect to the related FedAvg baselines, with values marked by an asterisk (*) indicating improvements over the FedAvg (Non-IID) baseline.The results show that CSCM significantly relieves performance degradation under Non-IID conditions. In particular, CSCM–YOLOv8 and CSCM–YOLOv9 achieve mAP50 improvements of 37.5% and 40.1%, respectively, while reducing communication cost by up to 81.1%, demonstrating a better accuracy communication trade-off.

Although the proposed CSCM framework realizes notable performance gains, the challenge of scale of large federated learning for dynamically participating clients might exist to some extent in extreme Non-IID or class-imbalanced scenarios. In addition, fixed rates for selecting clients and the similarity threshold can be challenging to set for different scenarios. In practice, the application of the proposed CSCM framework can be fit for purposes related to smart cities and surveillance applications for traffic surveillance and safe city scenarios where object detection models are trained on edge cameras with low-bandwidth communications.

E. ABLATION STUDY

To analyze the contribution of individual components in the proposed CSCM framework, we perform ablation analysis on three important dimensions: client selection rate, client filtering based on cosine similarity, and the choice of backbone networks as shown in Table 5. Firstly, by uniformly treating client selection rate, we observe that there indeed exists a trade-off between communication overhead and detection performance, and that a moderate client selection rate results in near-optimal performance and greatly reduced communication overheads. Secondly, by eliminating client filtering based on cosine similarity and adopting random client selection, we show that the performance indeed degrades, which confirms the effectiveness of our similarity-aware client selection in handling the problem of Non-IID in client datasets. Thirdly, we comparatively analyze and validate that the proposed CSCM framework generalizes well to different backbone networks and that YOLOv9-outperforming YOLOv8-in terms of both accuracy and reduced communication overhead due to its smaller and more efficient architectures.

To further examine the reliability of the proposed framework, we conducted a stability analysis using five independent experimental runs for each configuration. Each run used a different random seed and independent client data shuffling to reflect practical federated learning variability. The results are summarized in Table 6 as mean and standard deviation values.CSCM shows very low variation in precision, recall, and mAP@50 across runs. The standard deviation of mAP remains below 0.01 for both YOLOv8

TABLE 5. Ablation analysis of CSCM based FL.

Ablation Category	Configuration	Communication Cost (GB)	mAP50
Client Selection Rate	30%	1.42	0.801
	50% (Default)	2.09	0.829
	70%	3.61	0.832
	100% (FedAvg)	6.502	0.603
Client Selection Strategy	FedAvg (No selection)	6.502	0.603
	Random selection	3.88	0.691
	CSCM (Cosine similarity)	2.09	0.829
Backbone Architecture	CSCM–YOLOv8	2.09	0.829
	CSCM–YOLOv9	1.23	0.845

and YOLOv9. This indicates stable convergence under Non-IID settings. FedAvg shows higher variation, especially in recall and mAP, which suggests it is more sensitive to client data differences.The improved stability of CSCM comes from the cosine similarity based client selection. By filtering misaligned updates, the global model changes more smoothly during aggregation.Communication cost and computational complexity remain unchanged across runs since they depend on fixed configuration settings.These results confirm that CSCM provides not only better accuracy and lower communication cost, but also more consistent performance across multiple runs.

Table 5 presents the sensitivity analysis of the client selection mechanism. We varied the client participation fraction to 30%, 50%, 70%, and 100% to examine how it affects detection accuracy and communication cost. The results show a clear pattern. When fewer clients participate, the communication cost drops noticeably, but mAP@50 decreases slightly. As more clients are included, the accuracy improves modestly, yet the communication cost rises sharply. The 50% setting offers a reasonable balance. It maintains strong detection performance without introducing excessive communication overhead.

TABLE 6. Multi-run statistical results.

Model	Method	Precision	Recall	mAP@50	Comm. Cost (GB)	TCC (GFLOPs)
YOLOv8	FedAvg	0.828 ± 0.012	0.724 ± 0.015	0.806 ± 0.011	5.24	52360
YOLOv8	CSCM	0.741 ± 0.009	0.781 ± 0.010	0.829 ± 0.007	2.09	8378
YOLOv9	FedAvg	0.901 ± 0.010	0.744 ± 0.012	0.833 ± 0.009	3.06	30639
YOLOv9	CSCM	0.862 ± 0.008	0.759 ± 0.009	0.845 ± 0.006	1.23	4902

F. SCALABILITY DISCUSSION

To better understand how the proposed method would work in real settings, we discuss its scalability under larger client populations and stronger data heterogeneity. In large federated systems, adding more clients often increases variation in local updates. This can slow down convergence because client models may move in different directions. The cosine similarity based client selection in CSCM helps reduce this problem. It filters out updates that are not well aligned before aggregation.As the number of clients grows, this filtering step becomes more useful. It limits instability caused by highly diverse local data. Communication cost naturally increases when more clients participate in each round. However, by controlling the participation rate, the overall overhead can still be managed. In cases with stronger Non-IID distributions, the alignment based selection further stabilizes the global model and avoids severe drops in accuracy.Overall, CSCM is designed to handle practical federated environments with many heterogeneous clients while keeping convergence stable and communication cost under control.

G. EVALUATION OF MODEL PERFORMANCE FOR OD IN REAL TIME SYSTEMS

The results in Table 4 show the Hardware-Specific Inference Performance Comparison of Federated Models. Our federated YOLOv9c models achieve real-time object detection ($\geqslant$5 FPS) on resource-constrained devices, including the Jetson Nano and Jetson Xavier, when combined with TensorRT and FP16 precision. Compared to the baseline (non-optimized) federated models, which frequently fall below real-time thresholds, this is a notable improvement. Furthermore, the outcomes demonstrate that our framework is flexible in its deployment while retaining a respectable inference speed by accommodating varying hardware

TABLE 7. Hardware-specific inference performance comparison of federated models.

Device	Federated Model	Image Size	Precision	AIT(ms)	FPS
Raspberry Pi 5	Fed. YOLOv9c	640×640	FP32	940	1
	Fed. YOLOv9c + TRT	640×640	FP16	526	2
Jetson Nano	Fed. YOLOv9c	640×640	FP32	302	3
	Fed. YOLOv9c + TRT	640×640	FP16	166	6
	Fed. YOLOv9c	320×320	FP32	212	5
	Fed. YOLOv9c + TRT	320×320	FP16	146	7
Jetson Xavier	Fed. YOLOv9c	640×640	FP32	188	4
	Fed. YOLOv9c + TRT	640×640	FP16	94	8
	Fed. YOLOv9c	320×320	FP32	134	5
	Fed. YOLOv9c + TRT	320×320	FP16	62	11

TRT:TensorRunTime,AIT:Avg Inference Time,FPS:Frames Per Second

capabilities and image resolutions.All edge deployment experiments were performed with a batch size of 1 to reflect real-time inference conditions. Each model was executed with 20 warm-up iterations, followed by 100 inference runs, and the reported Average Inference Time (AIT) corresponds to the mean value across runs. The measured AIT and FPS include end-to-end processing, covering input preprocessing, model inference, and postprocessing (NMS and bounding-box decoding). TensorRT (FP16) results are reported using optimized ONNX engines. These enhancements confirm our suggested FL-based OD system's viability and effectiveness in actual edge computing settings.

The Jetson AGX Xavier consistently delivers superior detection accuracy and speed, while the Jetson Nano and Raspberry Pi 5 CPU perform well within their hardware limitations.

V. CONCLUSION

The proposed research integrates an advanced YOLO architecture, namely YOLOv8 and YOLOv9, with the FL framework to scalable OD challenges. Utilizing the CSCM ensures efficient client selection and model aggregation allowing the global model to absorb knowledge from diverse, Non-IID dataset while maintaining data privacy from data locality. The experimental evaluations on the dataset UniScenes, targeting a capture of the Non-IID dataset property of real-world FL setups, show that the devised approach is robust and agile enough. Important performance measurements, recall, and mAP show that CSCM-based outperformed conventional FedAvg significantly in reducing communication costs and computational overheads. The deployment phase further validates the practicality of the system, showing the adaptability of the optimized YOLO models on a range of edge devices, from resource-constrained Raspberry Pi CPUs to high-performance NVIDIA Jetson GPUs. The results show that FL enables the models to maintain high detection accuracy while optimizing inference speed and hardware efficiency, achieving real-time performance across various platforms. This study bridges the gap between centralized AI training and resource-efficient edge AI applications by converting trained YOLO models to ONNX and optimizing them with TensorRT. This study makes significant contributions to the field of FL-based OD. The framework adheres to the core principle of federated learning by training models locally without transmitting raw data, thereby supporting data locality and minimizing data exposure. Future work will focus on extending the proposed framework by integrating lightweight YOLO variants to further reduce computational and memory overhead, enabling efficient deployment on mobile and ultra resource constrained edge devices. In addition, the CSCM-based aggregation strategy can be expanded toward adaptive and dynamic client selection mechanisms that account for varying data distributions, system conditions, Differential Privacy mechanism and client availability across federated rounds. Another promising direction involves addressing cross device heterogeneity, including differences in hardware capability, communication bandwidth, and energy constraints, to enhance scalability and robustness in large-scale federated deployments.

REFERENCES

[1] J. Redmon, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.
[2] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.
[3] E. T. M. Beltrán, M. Q. Pérez, P. M. S. Sánchez, S. L. Bernal, G. Bovet, M. G. Pérez, G. M. Pérez, and A. H. Celdrán, "Decentralized federated learning: Fundamentals, state of the art, frameworks, trends, and challenges," *IEEE Commun. Surv. Tut.*, vol. 25, no. 4, pp. 2983–3013, 2023.
[4] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.
[5] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. CVPR*, vol. 1, Dec. 2001, pp. I-511–I-518.
[6] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Mali, Jun. 2014, pp. 580–587.
[7] R. Girshick, "Fast R-CNN," 2015, *arXiv:1504.08083*.
[8] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, Jun. 2017.
[9] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," 2018, *arXiv:1804.02767*.
[10] D. Wu, S. Lv, M. Jiang, and H. Song, "Using channel pruning-based YOLO v4 deep learning algorithm for the real-time and accurate detection of apple flowers in natural environments," *Comput. Electron. Agricult.*, vol. 178, Nov. 2020, Art. no. 105742.
[11] O. E. Olorunshola, M. E. Irhebhude, and A. E. Evwiekpaefe, "A comparative study of YOLOv5 and YOLOv7 object detection algorithms," *J. Comput. Social Informat.*, vol. 2, no. 1, pp. 1–12, Feb. 2023.
[12] K. Vinoth and P. Sasikumar, "Lightweight object detection in low light: Pixel-wise depth refinement and TensorRT optimization," *Results Eng.*, vol. 23, Sep. 2024, Art. no. 102510.
[13] C. Y. Wang, I. H. Yeh, and H. Y. M. Liao, "YOLOv9: Learning what you want to learn using programmable gradient information," in *Proc. Eur. Conf. Comput. Vis.*, 2025, pp. 1–21.
[14] P. M. Mammen, "Federated learning: Opportunities and challenges," 2021, *arXiv:2101.05428*.
[15] S. Banabilah, M. Aloqaily, E. Alsayed, N. Malik, and Y. Jararweh, "Federated learning review: Fundamentals, enabling technologies, and future applications," *Inf. Process. Manage.*, vol. 59, no. 6, Nov. 2022, Art. no. 103061.
[16] L. Li, Y. Fan, M. Tse, and K. Y. Lin, "A review of applications in federated learning," *Comput. Ind. Eng.*, vol. 149, Jul. 2020, Art. no. 106854.
[17] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao, "A survey on federated learning," *Knowl.-Based Syst.*, vol. 216, Mar. 2021, Art. no. 106775.
[18] X. Yin, Y. Zhu, and J. Hu, "A comprehensive survey of privacy-preserving federated learning: A taxonomy, review, and future directions," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 1–36, Jul. 2022.
[19] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proc. IEEE*, vol. 105, no. 12, pp. 2295–2329, Dec. 2017.
[20] R. Fernandes, W. M. Rodrigues, and E. Barros, "Dataset and benchmarking of real-time embedded object detection for robocup ssl," in *Proc. Robot World Cup*, 2021, pp. 53–64.
[21] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. J. Lee, J. Ngiam, Q. V. Le, and Y. Wu, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 103–112.
[22] R. Hakani and A. Rawat, "Edge computing-driven real-time drone detection using YOLOv9 and NVIDIA Jetson nano," *Drones*, vol. 8, no. 11, p. 680, Nov. 2024.
[23] P. Sk, S. A. Kesanapalli, and Y. Simmhan, "Characterizing the performance of accelerated Jetson edge devices for training deep learning models," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 6, no. 3, pp. 1–26, 2022.

[24] K. Agarwal, A. Sanyo, S. Bakshi, M. Vinay, J. Jayapriya, and S. Deepa, "Performance analysis of YOLOv7 and YOLOv8 models for drone detection," in *Proc. Int. Conf. Netw., Multimedia Inf. Technol. (NMITCON)*, Sep. 2023, pp. 1–10.

[25] A. M. Elbir, B. Soner, S. Çöleri, D. Gündüz, and M. Bennis, "Federated learning in vehicular networks," in *Proc. IEEE Int. Medit. Conf. Commun. Netw. (MeditCom)*, Sep. 2022, pp. 72–77.

[26] J. Luo, X. Wu, Y. Luo, A. Huang, Y. Huang, Y. Liu, and Q. Yang, "Real-world image datasets for federated learning," 2019, *arXiv:1910.11089*.

[27] D. Jallepalli, N. C. Ravikumar, P. V. Badarinath, S. Uchil, and M. A. Suresh, "Federated learning for object detection in autonomous vehicles," in *Proc. IEEE 7th Int. Conf. Big Data Comput. Service Appl. (BigDataService)*, Aug. 2021, pp. 107–114.

[28] K. Doshi and Y. Yilmaz, "Federated learning-based driver activity recognition for edge devices," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2022, pp. 3337–3345.

[29] V. P. Chellapandi, L. Yuan, C. G. Brinton, S. H. Żak, and Z. Wang, "Federated learning for connected and automated vehicles: A survey of existing approaches and challenges," *IEEE Trans. Intell. Vehicles*, vol. 9, no. 1, pp. 119–137, Jan. 2024.

[30] S. Wang, C. Li, D. W. K. Ng, Y. C. Eldar, H. V. Poor, Q. Hao, and C. Xu, "Federated deep learning meets autonomous vehicle perception: Design and verification," *IEEE Netw.*, vol. 37, no. 3, pp. 16–25, May 2023.

[31] O. Urmonov, S. Sajid, Z. Aziz, and H. Kim, "Federated object detection scenarios for intelligent vehicles: Review, case studies, experiments and discussions," *IEEE Trans. Intell. Vehicles*, early access, Jun. 4, 2024, doi: 10.1109/TIV.2024.3408921.

[32] G. Rjoub, O. A. Wahab, J. Bentahar, and A. S. Bataineh, "Improving autonomous vehicles safety in snow weather using federated YOLO CNN learning," in *Proc. Int. Conf. Mob. Web Intell. Inf. Syst.*, Aug. 2021, pp. 121–134.

[33] F. Chi, Y. Wang, P. Nasiopoulos, and V. C. M. Leung, "Federated cooperative 3D object detection for autonomous driving," in *Proc. IEEE 33rd Int. Workshop Mach. Learn. Signal Process. (MLSP)*, Sep. 2023, pp. 1–6.

[34] B. A. M. Hashim and R. Amutha, "Elderly people activity recognition based on object detection technique using Jetson nano," *Wireless Pers. Commun.*, vol. 134, no. 4, pp. 2041–2057, Feb. 2024.

[35] J. Zhu, H. Feng, S. Zhong, and T. Yuan, "Performance analysis of real-time object detection on Jetson device," in *Proc. IEEE/ACIS 22nd Int. Conf. Comput. Inf. Sci. (ICIS)*, Jun. 2022, pp. 156–161.

[36] Z. Deng, B. Hao, G. Liu, X. Li, H. Wei, F. Huang, and S. Liu, "MonoMPV: Monocular 3D object detection with multiple projection views on edge devices," *IEEE Access*, vol. 12, pp. 136599–136612, 2024.

[37] B. Hariharan, P. Arbeláez, R. Girshick, and J. Malik, "Simultaneous detection and segmentation," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2014, pp. 297–312.

[38] S. Gidaris and N. Komodakis, "Object detection via a multi-region and semantic segmentation-aware CNN model," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1134–1142.

[39] K. S. Cherukuri, K. Sha, and Z. Huang, "Enabling federated object detection for connected autonomous vehicles: A deployment-oriented evaluation," 2025, *arXiv:2509.01868*.

[40] C. Quéméneur and S. Cherkaoui, "FedPylot: Navigating federated learning for real-time object detection in Internet of Vehicles," 2024, *arXiv:2406.03611*.

[41] T. Zheng, A. Li, Z. Chen, H. Wang, and J. Luo, "AutoFed: heterogeneity-aware federated multimodal learning for robust autonomous driving," in *Proc. 29th Annu. Int. Conf. Mobile Comput. Netw.*, Jul. 2023, pp. 1–15.

[42] G. K. Alshammari, A. Abubakar, N. M. O. S. Ahmed, and N. K. Alshammari, "Federated learning-based semantic segmentation for lane and object detection in autonomous driving," 2025, *arXiv:2504.18939*.

[43] A. Lakhan, T.-M. Grønli, P. Bellavista, S. Memon, M. Alharby, and O. Thinnukool, "IoT workload offloading efficient intelligent transport system in federated ACNN integrated cooperated edge-cloud networks," *J. Cloud Comput.*, vol. 13, no. 1, p. 79, Apr. 2024.

[44] K. Vinoth and P. Sasikumar, "Multi-sensor fusion and segmentation for autonomous vehicle multi-object tracking using deep q networks," *Sci. Rep.*, vol. 14, no. 1, p. 31130, Dec. 2024.

[45] W. A. Ezat, M. M. Dessouky, and N. A. Ismail, "Evaluation of deep learning YOLOv3 algorithm for object detection and classification," *Menoufia J. Electron. Eng. Res.*, vol. 30, no. 1, pp. 52–57, Jan. 2021.

[46] D. M. W. Powers, "Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation," 2020, *arXiv:2010.16061*.

[47] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, Apr. 2017, pp. 1273–1282.

[48] J. Konečný, H. B. McMahan, D. Ramage, and P. Richtárik, "Federated optimization: Distributed machine learning for on-device intelligence," 2016, *arXiv:1610.02527*.

[49] V. Smith, C.-K. Chiang, M. Sanjabi, and A. S. Talwalkar, "Federated multi-task learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 4427–4437.

[50] K. Vinoth and P. Sasikumar, "VINO_EffiFedAV: VINO with efficient federated learning through selective client updates for real-time autonomous vehicle object detection," *Results Eng.*, vol. 25, Mar. 2025, Art. no. 103700.

[51] C. Dwork and A. Roth, "The algorithmic foundations of differential privacy," *Found. Trends Theor. Comput. Sci.*, vol. 9, nos. 3–4, pp. 211–407, Aug. 2014.

[52] C. Quéméneur and S. Cherkaoui, "Fedpylot: Navigating federated learning for real-time object detection in Internet of Vehicles," *arXiv:2406.03611*.

[53] B. Wang, D. Feng, J. Su, and S. Song, "An effective federated object detection framework with dynamic differential privacy," *Mathematics*, vol. 12, no. 14, p. 2150, Jul. 2024.

[54] Y. Ge and J. Zhou, "Blockchain based federated learning for object detection," in *Proc. 2nd Int. Conf. Control Intell. Robot.*, Jun. 2022, pp. 608–612.



K. VINOTH received the M.Tech. degree in VLSI and embedded systems from Pondicherry University and the Ph.D. degree from Vellore Institute of Technology (VIT), Vellore. He is currently an Assistant Professor with over eight years of teaching and research experience. His work focuses on developing efficient and real-time AI solutions for autonomous systems and smart applications. He has published several research works and actively contributes to academic and research activities. He has also trained a large number of students in emerging technologies and guided projects in AI, the IoT, and real-time systems. His research interests include computer vision, edge computing, deep learning, object detection, multi-object tracking, federated learning, and generative AI.



SASIKUMAR PERIYASAMY (Senior Member, IEEE) received the B.E. degree in electrical and electronics engineering from Anna University, the M.E. degree in applied electronics from the Government College of Technology (an autonomous institution), and the Ph.D. degree in QoS enhancement for distributed clustering techniques in wireless sensor networks from Vellore Institute of Technology, Vellore. He is currently a Professor with Vellore Institute of Technology. He is also an expert in embedded systems, wireless sensor networks, communication engineering, and the IoT.